Basic (embeddings as geometry) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `embedding_similarity_practice.py`: "embeddings are just numbers that capture meaning" stays an abstract claim until you embed a few sentences yourself, compute the distance between each pair, and watch 2 similar sentences actually score closer together than unrelated ones — this is the exercise that turns it into something you measured yourself. **If not:** every later exercise in this document (chunking, retrieval, the Build Task) would be built on a formula you took on faith instead of one you watched work.

Solution

```
# embedding_similarity_practice.py
import math
from itertools import combinations
from openai import OpenAI

client = OpenAI()

SENTENCES: list[str] = [
    "The cat sat on the mat.",
    "A feline rested on the rug.",
    "Stock prices rose today.",
    "It might rain tomorrow.",
    "The bakery sells fresh bread every morning.",
]

def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
```

```
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

# Embed every sentence once.
embeddings = {}
for sentence in SENTENCES:
    embeddings[sentence] = get_embedding(sentence)

# Score every pair of sentences, then sort most-similar first.
scored_pairs = []
for sentence_a, sentence_b in combinations(SENTENCES, 2):
    score = cosine_similarity(embeddings[sentence_a], embeddings[sentence_b])
    scored_pairs.append((score, sentence_a, sentence_b))

scored_pairs.sort(reverse=True)
for score, sentence_a, sentence_b in scored_pairs:
    print(f"{score:.4f}  {sentence_a!r} <-> {sentence_b!r}")
```

Expected output: the same 10 pairs as Basic, but sorted, so the cat/feline pair prints first, with the highest score, instead of being buried wherever the loop happened to reach it.